

Mark as done

Overview

In this coursework, you will design and implement a convolutional neural network (CNN) **from scratch**, using only the **Python standard library**, **NumPy**, **OpenCV**, and **Matplotlib**. Your task is to develop a model that learns to reconstruct images from shuffled patches, training it to correctly rearrange scrambled inputs into their original order.

Task

1. Dataset Preparation

- Input images will be resized to **32 x 32 pixels**.
- Each image will be divided into **4 x 4 non-overlapping patches** (each patch is 8 x 8 pixels).
- The patches will then be **randomly shuffled** to produce a scrambled version of the image.

2. Model Objective

- Build a CNN that takes as input the scrambled version of the image.
- Train the model to **predict the correct spatial arrangement of patches** and reconstruct the original image.

3. Implementation Requirements

- Implement all core components of the CNN **from scratch**: convolutional layers, activation functions, pooling, loss, and training loop.
- You may use NumPy for matrix operations, OpenCV for image preprocessing, Albumentations for data augmentation, and Matplotlib for visualisation.
- **No deep learning frameworks (e.g., TensorFlow, PyTorch, Keras, JAX, OneFlow) are allowed. The use of any deep learning framework will result in a grade of ZERO for the coursework.**

4. Training & Evaluation

- To train your model, use the training dataset provided in Week 1, [acv_train_32x32.zip](#), which is available for download [here](#).
- Evaluate the model's ability to reconstruct scrambled images and measure its accuracy or reconstruction quality.
- Visualize both input (scrambled) and output (reconstructed) images for qualitative assessment.

Deliverables

- **Source code**: A well-documented Python implementation of your CNN.
- **Model weights**: To test your model, we will download your weights and perform an inference on a held-out dataset. For more information on our testing process, please refer to the 'Submission instructions and testing procedure' section below.
- **Report (max. one page, double column)**:
 - The report should be written in LaTeX with Overleaf, and attached to the submission as a PDF.
 - The report should include:
 - A description of your CNN architecture and design choices.
 - Training procedure (loss function, optimiser, number of epochs, data augmentation, etc.).
 - Evaluation protocol, metrics and results.
 - Visual examples comparing shuffled vs. reconstructed images.
 - Reflections on challenges faced and possible improvements.

Grading criteria

- **Model Evaluation [70 marks]**:
 - **Objective**: To attain the lowest possible generalisation error on a held-out test set.

- **Held-out test set:** To ensure a fair evaluation of the performance of your model, you will not have access to this test set. However, similarly to the provided training set, the held-out test set consists of 32x32-pixel images selected containing handwritten digits.
- **Performance evaluation on the held-out test set:** Model performance will be evaluated based on how accurately your model reconstructs the patch arrangement. For each test image, we calculate the percentage of patches placed in their correct original positions. The final *accuracy* metric is the average of these percentages across all test images.
 - **Note:** Input images may contain multiple all-black patches. Since these patches are identical, it is impossible to determine their exact original indices. To avoid unfair penalisation, any patch where all pixel values are below 0.05 (on a 0.0–1.0 scale) will be excluded from the accuracy calculation.
- **Marks:** the marks for this part (i.e. Model Evaluation) will be directly obtained from the *accuracy* of your model computed on the held-out test set:
 - An accuracy of **<30%** will receive **0 marks** for this part of the coursework.
 - An accuracy **>=30% and <40%** will receive **35 marks** for this part of the coursework.
 - An accuracy **>=40% and <50%** will receive **42 marks** for this part of the coursework.
 - An accuracy **>= 50% and <60%** will receive **50 marks** for this part of the coursework.
 - An accuracy **>=60%** will receive **70 marks**, i.e. full marks for this part of the coursework.
- **Report [30 marks]:**
 - Description of the CNN architecture and design choices **[6 marks]**.
 - Training procedure (loss function, optimiser, number of epochs, data augmentation) **[6 marks]**.
 - Evaluation protocol, metrics and results **[6 marks]**.
 - Visual examples comparing shuffled vs. reconstructed images **[6 marks]**.
 - Reflections on challenges faced and possible improvements **[6 marks]**.

Submission instructions and testing procedure

You should submit a zip file containing a directory named with your *k number*, e.g. *k1234567*. Inside that directory, you should provide the source code, model weights, and report. There is a limit of 500MB for the submission file, which you should take into account when you develop your model.

Inside the directory named with your *k number*, you must provide a Python script called **test.py**, and a model called **final.pkl**. To test the performance of your model, your code will be executed (from within the directory named with your *k number*) as follows:

```
$ python test.py --test-dir <test_dir_path> --model final.pkl
```

Inside the testing directory there are going to be images in PNG format, each of them with a resolution of 32 x 32 pixels. However, these images, will be shuffled (i.e. they come from images that were divided into 4 x 4 patches and shuffled).

The job of your **test.py** script is to predict the original position of each patch from the shuffled image. For each image, your script should output a line with the following syntax (in row-major order): **image_name.png,p_0,p_1,p_2,...,p_15**

Here, **p_0** is the **original index** (from 0 to 15) of the patch that is currently at position 0 (top-left) in the **shuffled image**. Similarly, **p_1** is the original index of the patch at position 1, and so on.

For example, given the following mapping:

- **Original image grid (correct order):**

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

- **Shuffled image grid (input to your model):**

15	14	13	12
11	10	0	1
2	3	4	5
6	7	8	9

The correct output line would be: **1.png,15,14,13,12,11,10,0,1,2,3,4,5,6,7,8,9**

This is because the patch at position 0 in the shuffled grid is the one that belongs at position 15 in the original; the patch at position 1 is the one that belongs at position 14, and so on.

The order of the lines in the output of your test script is irrelevant; however, it is crucial that your script outputs as many lines as there are images in the test directory. In other words, **you must provide a solution for every single test image. If your test script fails to do so, you will receive zero marks.**

Frequently asked questions

Q1: What architecture and loss function should be used?

R1: This decision is part of the assignment, and we expect you to choose an appropriate combination and provide a clear justification for it in your report. Please consider the problem at hand and the resources available in Colab (e.g., computational power, memory constraints) when making this selection.

Last modified: Monday, 26 January 2026, 5:50 PM

◀ [Training set used for both, Python Exercises and Coursework](#)

Jump to...

[Coursework Submission Link ▶](#)



Key links

[Terms of Use](#)

[Privacy Policy](#)

[Contact Us](#)

[Student Data Collection Notice](#)

[Accessibility Statement](#)

[Release Notes](#)

Social media



© 2026 King's College London, Strand, London WC2R 2LS, England, United Kingdom

Tel [+44 \(0\)20 7836 5454](tel:+442078365454)